BrowserStack Demo Store

Test Design Approach & Coverage Rationale

Companion document to BStackDemo_TestCases_Final.xlsx

Prepared by: QA Engineering | Application: https://bstackdemo.com/ | Version: 1.0

1. Purpose

This document explains the test design techniques used to build the 131 test cases in the accompanying Excel workbook (BStackDemo_TestCases_Final.xlsx), covering all 10 user stories defined in the Business Requirements Specification (SRS) for the BrowserStack Demo Store. For each technique, this document states why it was selected, which test cases (TC_IDs) were derived using it, and the assumptions made during test case design.

2. Coverage Summary

Total test cases: 131, spanning positive, negative, boundary, validation, error-handling, UI, functional, integration, and edge-case scenarios across all 10 user stories.

User Story	Feature Area	Test Cases
UserStory_01	User Authentication (Sign In & Sign Out)	20
UserStory_02	Navigation & Global Header	12
UserStory_03	Product Catalogue Browsing	14
UserStory_04	Product Filtering by Vendor and Category	14
UserStory_05	Product Sorting	9
UserStory_06	Favourites (Wishlist)	8
UserStory_07	Shopping Cart Management	15
UserStory_08	Checkout – Shipping Information	19
UserStory_09	Order Confirmation	10
UserStory_10	Location-Based Product Offers	10

Priority	Test Cases
High	73
Medium	46
Low	12

3. Test Design Techniques

3.1 Equivalence Partitioning (EP)

Why it was used: Many fields in the application (Username, Password, checkout fields, filter/sort selections) accept a defined set of valid values alongside an effectively infinite set of invalid ones. Rather than testing every possible

input, inputs were grouped into equivalence classes — valid demo accounts, invalid/absent credentials, valid alphanumeric address text, and empty/special-character-only text — with one representative case tested per class.

- **Applied to:** Login credential classes (TC_004–TC_013), checkout field content classes such as alphanumeric text, numeric post codes, and special-character-only input (TC_100–TC_103), and filter/vendor selections (TC_048–TC_053)
- **Assumption:** All five demo usernames (demouser, image_not_loading_user, existing_orders_user, fav_user, locked_user) behave identically for generic authentication flows except where the SRS calls out unique behaviour (image loading, pre-existing data, account lock), so each unique behaviour was tested individually while generic flows used demouser as the representative valid class.

3.2 Boundary Value Analysis (BVA)

Why it was used: Boundary analysis targets the edges of input ranges and data volumes, where defects are statistically most likely to occur — minimum/maximum text length, single vs. many items, and tie conditions in sorted data.

- **Applied to:** TC_032 (long username in header), TC_046 (long product name), TC_069 (single-item sort/filter result), TC_077 (favouriting many products), TC_091 (cart with a large number of distinct products), TC_104–TC_105 (minimum 1-character and maximum 200-character First Name input), TC_106 (numeric post code boundary)
- **Assumption:** Exact maximum field-length limits are not specified in the SRS, so a 200-character string was used as a practical stress value for text inputs; actual limits should be confirmed against the live application and this test adjusted accordingly.

3.3 Decision Table Testing

Why it was used: Several features are governed by combinations of independent conditions that must be evaluated together — for example, authentication state × field completeness during login, and multiple simultaneous filter selections. Decision tables ensure every meaningful combination of conditions and their resulting actions is covered, not just each condition in isolation.

- **Applied to:** Login validation combinations — username present/absent × password present/absent (TC_010–TC_012); multi-filter OR-logic combinations within the Brand group and combined Brand+Category filtering (TC_051, TC_053); the empty-result combination where filters match zero products (TC_056)
- **Assumption:** Filter groups combine with AND logic across groups and OR logic within a group, as stated in the SRS conversation notes for UserStory_04; this rule was used to determine expected results for every filter-combination test case.

3.4 State Transition Testing

Why it was used: The application has clearly defined states with triggered transitions — unauthenticated → authenticated → logged out, empty cart → populated cart → empty cart (post-checkout), and un-favourited → favourited → un-favourited. Testing the transitions themselves (not just the states) catches defects in state persistence, session clearing, and toggle behaviour.

- **Applied to:** Authentication lifecycle: login, session persistence across navigation, logout, and re-login (TC_004, TC_015, TC_016, TC_020); protected-route redirection after logout (TC_018–TC_019); cart lifecycle from empty → populated → checked out → cleared (TC_080, TC_085, TC_109); favourite toggle on/off (TC_072)
- **Assumption:** Session state (authentication, cart, favourites) is scoped to the browser session and is not expected to persist after logout or across devices, consistent with the SRS statement that the cart and favourites reset upon logout.

3.5 Error Guessing

Why it was used: Experience-based testing was applied to anticipate failure modes not explicitly documented in the SRS but common to web applications of this type — network interruptions, permission denials, malformed input, and data-load failures. These scenarios rely on QA judgement rather than a formal derivation technique.

- **Applied to:** TC_045 (product data load failure), TC_103 (special-character-only First Name), TC_111 (checkout network/server failure), TC_120 (receipt download failure), TC_130 (geolocation permission denied), TC_131 (geolocation unavailable)
- **Assumption:** The application is assumed to fail gracefully (user-facing error messaging, no unhandled exceptions) for these scenarios, since the SRS does not define explicit error-handling behaviour for network or permission failures; expected results describe the minimally acceptable graceful-failure behaviour rather than an exact error message.

3.6 Pairwise Testing

Why it was used: The product listing page allows filters and sort order to be combined freely. Testing every possible combination of every filter value with every sort option would be exhaustive and low-value; pairwise testing selects a representative set of two-way combinations (one filter paired with one sort order) that still exercises the interaction logic between the two features.

- **Applied to:** TC_057 (Samsung filter + Lowest-to-Highest sort), TC_058 (Apple filter retained when sort order changes), TC_065 (Google filter + ascending sort), TC_066 (Apple filter retained across a sort change)
- **Assumption:** Filter and sort are independent, orthogonal features per the SRS ('Filter is independent of sort order'), so a small representative sample of filter/sort pairs is sufficient to validate the interaction without needing to test every filter value against every sort option.

4. General Assumptions

- The default password 'testingisfun99' is valid for all demo accounts, as stated in the SRS; an 'invalid password' negative case uses any other selectable value from the Password dropdown.
- Exact pixel-level responsive breakpoints are not specified in the SRS; UI/responsiveness test cases (e.g. narrow-viewport reflow) validate the general behaviour described (no overlap, no broken layout) rather than a specific breakpoint value.
- Where the SRS references an element by ID (e.g. login-btn, firstNameInput, confirmation-message), those IDs were used as the source of truth for locating elements during test execution.
- 'Actual Result' and 'Status' columns are intentionally left as 'Not Executed' / blank, since this workbook defines planned test coverage prior to a test execution cycle.
- Test cases assume the application is exercised in a standard desktop browser unless a test case explicitly targets responsive/narrow-viewport behaviour.

5. Traceability

Every test case in the workbook includes a User Story column mapping it back to the corresponding SRS user story (UserStory_01 through UserStory_10), enabling full bidirectional traceability between business requirements, acceptance criteria, and executable test cases.